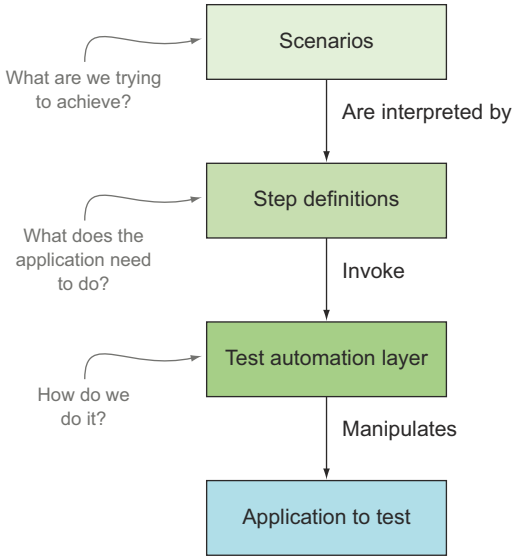


Figure 6.3 A well-designed test automation suite has a number of layers.

For example, you could implement the step that defines the flying distance by directly injecting data into a test database, by calling a web service, or via the user interface. But the step definition shouldn't need to worry about such details; its job is to set up the test data described in the scenario. You might do this as shown here:

Configure the test database through a dedicated API. ❶

```
TestDatabaseAPI inTheTestDatabase;

@Given("the flying distance between $departure and $destination is
       $distance km")
public void defineTheFlyingDistanceForATrip(String departure,
                                            String destination,
                                            int distance) {
        inTheTestDatabase.theDistanceBetween(departure)
                         .and(destination)
                         .is(distance);
}
```

Use this API to set up the test data. ❷

Inject the test data into the database. ❸

In this example, the step definition method interprets the text in the feature file ❷ and uses a dedicated class ❶ to configure the test database ❸. The step definition method doesn't care how it's done; it just describes the action it needs to achieve. The application-specific library worries about the rest.

Using a layered architecture like this is a key part of writing maintainable, readable, automated acceptance tests. You'll see much more of this approach in the coming chapters, particularly in chapter 7.

6.2 Implementing step definitions: general principles

There are many BDD tools you can use to automate scenarios like the ones we've been looking at, using different languages and often targeting different environments. The choice of which tool is right for your team will depend on how comfortable the team is